mathematics

MDPI

Article

An Effective Method for Slicing Triangle Meshes Using a Freeform Curve

Seung-Yong Lee, Seong-Hyeon Kweon and Seung-Hyun Yoon *

Department of Multimedia Engineering, Dongguk University, Seoul 04620, Republic of Korea; randysylee@dongguk.edu (S.-Y.L.); kshyunee@dgu.ac.kr (S.-H.K.)
* Correspondence: shyun@dongguk.edu

Abstract: Slicing 3D polygonal meshes is a fundamental operation in various applications such as virtual surgery, garment simulation, and game development. Existing methods primarily slice meshes using either a single line or a set of line segments approximating a smooth curve. This paper introduces a novel approach to freely slice a triangle mesh using a freeform curve without discretizing it into line segments. The user draws a stroke on the screen, defining the desired cutting trajectory. Subsequently, a freeform curve approximating this stroke is generated and extended into a ruled surface in the user's viewing direction. To efficiently compute intersections between the ruled surface and a triangle mesh, the Line–Surface Intersection (LSI) problem is broken down into two subproblems: Plane–Curve Intersection (PCI) followed by Line–Line Intersection (LLI). Intersection points are then connected to form polylines, effectively cutting the mesh into multiple submeshes. To ensure the solidity of the submeshes, cross-sections are generated by trimming the ruled surface along the polylines and merged with the corresponding submeshes. Our method empowers users to slice triangle meshes along arbitrary trajectories encompassing both straight and freely curved paths while preserving efficiency and accuracy. The effectiveness of the proposed approach is demonstrated through experimental results showing various examples of mesh slicing.

Keywords: mesh processing; mesh slicing; freeform curve; ruled surface; line–surface intersection

MSC: 68U05

check for updates

Citation: Lee, S.-Y.; Kweon, S.-H.; Yoon, S.-H. An Effective Method for Slicing Triangle Meshes Using a Freeform Curve. *Mathematics* **2024**, *12*, 1432. https://doi.org/10.3390/math12101432

Academic Editor: Shiaofen Fang

Received: 13 March 2024
Revised: 1 May 2024
Accepted: 5 May 2024
Published: 7 May 2024

Copyright: © 2024 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Polygonal meshes are widely used in computer graphics and geometric modeling for their simplicity and effectiveness in representing three-dimensional shapes. Over the past few decades, various techniques [1–4] have been developed for polygon mesh processing (PMP), including remeshing, smoothing, decimation, deformation, and more. Mesh slicing refers to dividing a mesh into multiple submeshes, and numerous algorithms have been developed for this purpose; however, most methods rely on user-specified lines for slicing polygonal meshes, with less focus on using freeform curves. In this paper, we propose an effective method for slicing solid-type triangle meshes using a freeform curve, assuming that the resulting sliced submeshes are also solid-type.

Slicing a triangle mesh with a single line is a relatively straightforward process, essentially reduced to solving the intersection problem between triangle edges and the slicing plane formed by extruding the line in the viewing direction (refer to Figure 1a,b). By substituting the parametric line equations $l(t) = (x(t), y(t), z(t))$ of triangle edges into the implicit plane equation $ax + by + cz + d = 0$, the intersection points can be easily computed. Subsequently, the triangle mesh is cut by the polylines connecting these intersection points, resulting in multiple submeshes with open boundaries (holes). Each boundary is then filled with the cross-section formed by hole-filling or a simple triangulation method. Figure 1b shows the result of slicing the Bunny model using a line, with both resulting submeshes being solid. While plane-based mesh slicing is straightforward and simple, it can lead to

undesired results due to the infinite extension of the slicing plane boundary. To address this limitation, Lee et al. [5] proposed a method using a bounded plane to selectively slice only the desired portion. However, even when using a bounded plane, slicing into submeshes with complex shapes requires significant user intervention.

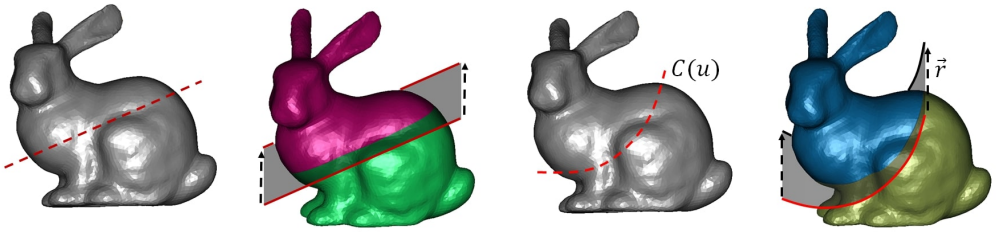


Figure 1. Mesh slicing types: (**a**) a straight line, (**b**) slicing result by (**a**), (**c**) a freeform curve, (**d**) slicing result by (**c**).

Mesh slicing with a freeform curve offers users greater flexibility and potential; however, existing methods [6–8] take a simple approach to discretizing a freeform curve into a set of line segments and applying a line-based method, as shown in Figure 2b. This approach inevitably leads to approximation errors and generates an excessive number of unnecessary boundary edges, as shown in Figure 2d. In this paper, we propose a new method for mesh slicing using a freeform curve without discretizing it into line segments. We directly compute the intersection points between a freeform curve and triangles (see Figure 2a) and cut the triangles by connecting these intersection points, resulting in optimal boundary edges (see Figure 2c).

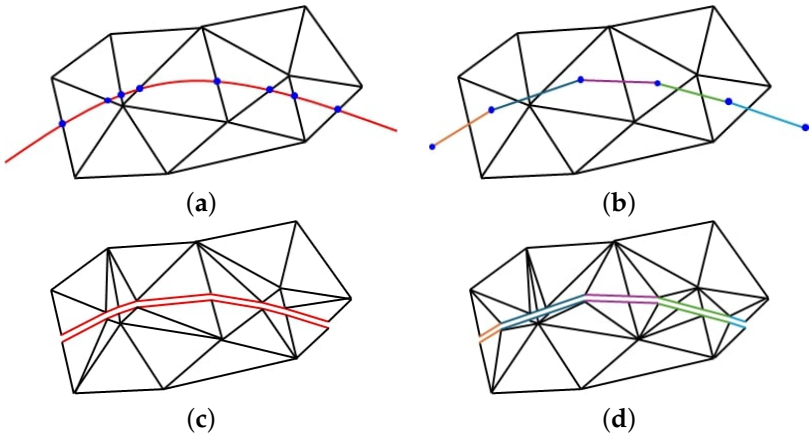


Figure 2. Mesh cutting with a freeform curve: (**a**) intersection points between triangles and a freeform curve, (**b**) discretizing a freeform curve into line segments, (**c**) result of cutting by intersection points, and (**d**) result of cutting by discrete line segments.

Our method starts with a two-dimensional curve $C(u)$ and extends it to a ruled surface $R(u,v) = C(u) + v\vec{r}$ by linearly extruding it in the user's viewing direction $\vec{r}$ (see Figure 1c,d). Then, we compute the intersection points between edges of triangles and the ruled surface, which is reduced to a Line–Surface Intersection (LSI) problem. In general, for a parametric surface $S(u,v) = (x_1(u,v), y_1(u,v), z_1(u,v))$ and a line $l(t) = (x_2(t), y_2(t), z_2(t))$, the LSI problem can be formulated as follows:

$$\begin{cases} x_1(u,v) &= x_2(t) \\ y_1(u,v) &= y_2(t) \\ z_1(u,v) &= z_2(t) \end{cases}. \tag{1}$$

By eliminating the linear variable t of $l(t)$, we can obtain a system of nonlinear equations with two unknowns u and v, as follows:

$$\begin{cases} f(u,v) &= 0 \\ g(u,v) &= 0 \end{cases}. \tag{2}$$

This system can be efficiently solved by guessing an initial solution (u_0, v_0) through iterative subdivision of the parametric domain, then applying the Newton–Raphson method [9]. However, this approach may incur significant computational costs when dealing with a large number of lines, i.e., edges of all triangles.

In this paper, we confine the LSI problem to a ruled surface $R(u,v)$ and propose an efficient algorithm for calculating the intersection points with a line $l(t)$. Our method decomposes the LSI problem of a ruled surface into two subproblems: Plane–Curve Intersection (PCI) followed by Line–Line Intersection (LLI). First, an intermediate intersection point $C(u^*)$ is determined by solving the PCI problem. Subsequently, the final intersection point $R(u^*, v^*)$ is computed by solving the LLI problem. Similar to the case with the line, the triangle mesh is cut by the polylines connecting intersection points. When using a curve, the cross-section is not planar anymore, and should be a subregion of the ruled surface $R(u,v)$. As all intersection parameters are computed within the parameter domain of $R(u,v)$, it is relatively straightforward to generate the cross-section by cutting the ruled surface. Figure 1d shows the result of slicing the Bunny model using a freeform curve. The overall process of the proposed method is illustrated in Figure 3.

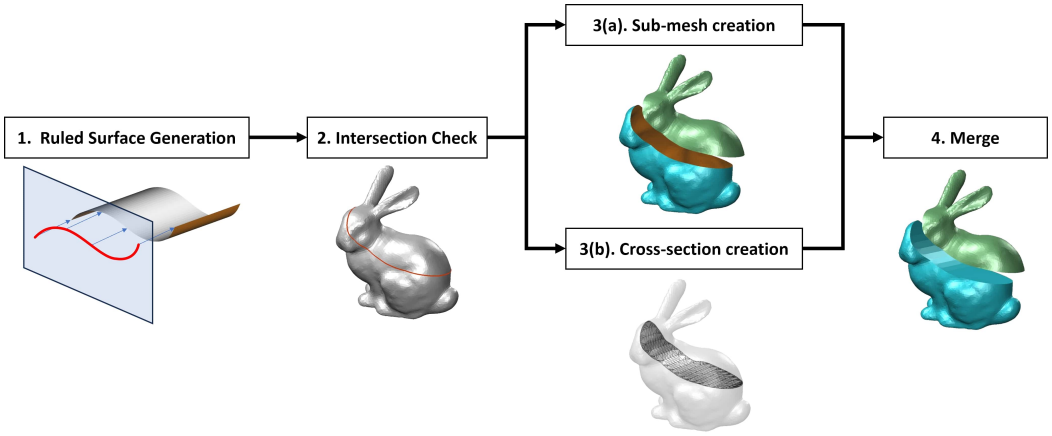


Figure 3. Overview of the proposed method.

Our method has several advantages, which can be summarized as follows:

- **Effectiveness.** The proposed method enables an effective slicing of a given triangle mesh into multiple submeshes both for straight strokes and for highly complex strokes provided by the user.
- **Flexibility.** Our method provides easy control over the resolution of the generated cross-sections by adjusting the number of sampling points on the ruled surface.
- **Efficiency and robustness.** The LSI problem is decomposed into two subproblems, first the PCI problem and then the LLI problem. This decomposition reduces the computational cost of calculating intersection points while simultaneously improving the robustness of the proposed method.

The remainder of this paper is organized as follows. In Section 2, we provide a brief review of previous works on surface mesh slicing. Section 3 describes the mesh slicing algorithm using a freeform curve. Various experimental results and potential applications are presented in Section 4, then we discuss the results, conclude the paper, and propose future work in Section 5.

2. Related Work

In this section, we examine previous research directly related to the contributions of this paper, particularly including mesh slicing techniques and the computation of intersections among diverse freeform models. Similar to the line-based approach mentioned in Section 1, most methods involve the step of creating a cutting plane to slice a 3D object. Chu et al. [10] introduced a method for creating a cutting plane to slice large models that exceed the general 3D printer's printable size. They used a user-specified line to define the cutting plane, allowing the user to partition the model for 3D printing. Kang and Kim [11] explored the process of cutting a 3D garment model into patches. Their method overcomes the limitations of slicing using only a line by allowing for user-defined cutting lines composed of multiple line segments. If the cutting line is a single segment, a traditional slicing plane is generated; however, for complex lines, users must connect them, creating a projected 2D polygon on the screen. Subsequently, triangles are intersected with either a plane or a polygon to define the patches. This approach is limited to cutting only the front side of the mesh, whereas our method supports completely cutting the mesh into submeshes. Owada et al. [12] presented a method where user strokes are converted into straight lines approximating them. A slicing plane is then created by extruding these lines in the viewing direction. These methods have the common limitation of approximating the user's strokes using a series of line segments, resulting in reduced slicing freedom and lower precision. To overcome this limitation, Mitani [6] proposed a slicing algorithm in which the user defines a cutting path with freehand strokes on the screen. This method performs intersection checks in 2D by projecting the polygonal mesh onto the screen, and prioritizes simplicity and robustness over precision by employing techniques such as node-snapping, vertex-to-line movement, and path simplification. While this approach offers ease of use and prevents degenerate triangles that could cause computational errors, the significant modifications to the stroke can compromise the precision of the slicing boundary.

Owada et al. [8] presented a slicing algorithm similar to our approach, featuring an interactive interface for volumetric mesh slicing visualized with a polygonal mesh to reveal the inner texture. A user-drawn freeform stroke is sampled into line segments and swept in a direction orthogonal to the screen, generating a curved discrete plane that slices the volumetric mesh through a constructive solid geometry (CSG) operation. This method then parameterizes the cross-section to visualize the inner texture of the polygonal mesh. Although this addresses the freedom of stroke issue, the issue of discretizing the freeform stroke remains.

Several methods [13–15] have explored virtual cutting tools for mesh manipulation. Khan et al. [13] presented a method for cutting polygonal meshes. Their approach removes faces that intersect with the slicing plane defined by a virtual scalpel blade. Subsequently, the boundary is regenerated by connecting the intersection points to the boundaries of the remaining faces. For generating a cross-section, they refined an initial triangulation through a multi-level refinement process. Kamarianakis et al. [14] addressed potential mesh subdivision issues during virtual scalpel interactions. When an intersection with a tetrahedral polygon is detected, the intersecting vertex is projected onto the scalpel's path to prevent unnecessary subdivision. The mesh is subsequently separated along this projected path. Caligiana et al. [15] employed a haptic device (Phantom) to facilitate interaction with a virtual surgical tool for mesh slicing, where intersecting triangles are subdivided using a longest-edge refinement strategy. While these virtual tool approaches offer real-time interaction and may be well-suited for specific applications, they could require additional hardware or introduce complexities in tool manipulation. In contrast, our stroke-based approach prioritizes user intuition and ease of use while maintaining efficient mesh slicing functionality.

Mesh slicing algorithms can be categorized based on the intersection types involved: discrete-discrete (plane–plane, plane–line, line–line), discrete-freeform (plane–surface, plane–curve, line–surface, line–curve), and freeform-freeform (surface–surface, curve–surface, curve–curve). Our method can be classified as a discrete-freeform case, wherein

a triangle intersects with a ruled surface. Lu et al. [16] discretized the surface into square patches and analyzed the signs of each patch's corner vertices by substituting the parameters into the implicit plane equation. If opposite signs are detected on an edge, this indicates that an intersection point lies on that edge. Sharma et al. [17] proposed a tracing method for finding the intersection between a parametric Bézier triangular surface and a plane, specifically focusing on triangular domains. Their method utilizes the parametric equations of the Bézier surface and the plane equation to efficiently compute their intersection points. The tracing method starts from an initial intersection point and iteratively follows the intersection curve until it reaches a boundary or meets a predefined stopping condition.

Li et al. [18] introduced a method for intersecting a ruled surface with a plane, similar to our approach. Their method involves discretizing the ruled surface by sampling the directrix curve and generating line segments along the generator direction to find intersections with the plane; however, unlike our approach, this suffers from the drawback of discretizing the ruled surface. Beer et al. [19] explored iterative approaches to solving the Line-Surface Intersection (LSI) problem, which entails finding the closest point on the surface to a given point on the line, then presented alternative solutions to the LSI. Martin et al. [20] proposed a numerical approach to the LSI problem, framing it as a ray tracing problem. A ray is considered as the intersection line between two planes with normals perpendicular to the ray's direction and passing through the ray's origin. By substituting the surface point into the implicit equations of these two planes, the system of equations is solved using Newton's method. Busé et al. [21] introduced a matrix-based implicit representation for Bézier patches (MRep) to solve the LSI problem, while Shen et al. [22] tackled degenerate cases in the general MRep method, resulting in higher accuracy.

3. Mesh Slicing Using a Freeform Curve

3.1. Ruled Surface Generation

This section outlines the process of generating a ruled surface from the user's freehand stroke on the screen. A ruled surface, which is a specific kind of freeform surface, is considered akin to a curved plane due to its planarity (having zero Gaussian curvature). Figure 4 illustrates an example of a user's freehand stroke (depicted in black) on the screen. Points are sampled at regular intervals along the stroke. For each sampled point $\mathbf{s}_i$, a ray $r(t)$ is cast in the viewing direction to calculate the intersection points $\mathbf{p}_i$ and $\mathbf{q}_i$ with the bounding planes π_1 and π_2 of a triangle mesh, as demonstrated in Figure 4. A cubic B-spline curve $C(u)$ is then constructed to approximate the intersection points $\mathbf{p}_i$ on π_1 within an error bound ϵ. Subsequently, the parametric ruled surface is defined as

$$R(u, v) = C(u) + v\vec{r}, \quad \text{for } 0 \leq u, v \leq 1, \tag{3}$$

where $\vec{r} = \mathbf{q}_i - \mathbf{p}_i$ represents the constant directional vector of the generator of $R(u, v)$. Figure 4 shows the ruled surface (in red) generated using the proposed method. While a ruled surface exhibits planar characteristics, it cannot be represented by an implicit function such as a plane; therefore, slicing a mesh with a ruled surface requires more computational steps compared to using a plane. Our technique requires the user's stroke to intersect the mesh fully, accommodating both freeform curves and conventional straight lines.

3.2. Ruled Slicing

When the ruled surface $R(u, v)$ has been generated from the user's freehand stroke, the next step involves computing the intersection lines between the ruled surface and the triangles of a mesh to slice the mesh effectively. We refer to this process as *ruled slicing*. Because only a subset of the mesh's triangles intersect with the ruled surface, identifying these candidate triangles is crucial for improving computational efficiency. To facilitate this, we tessellate the ruled surface into triangles and construct its Bounding Volume Hierarchy (BVH), then carry out intersection tests between the mesh triangles and the constructed BVH. For our study, we employed a straightforward Axis-Aligned Bounding Box (AABB)

as the BVH type. Figure 5 illustrates the triangles (highlighted in yellow) of the Bunny model that intersect with the ruled surface, identified using the BVH of $R(u, v)$.

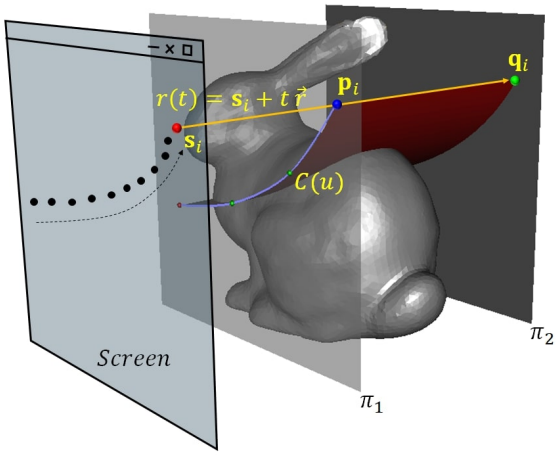
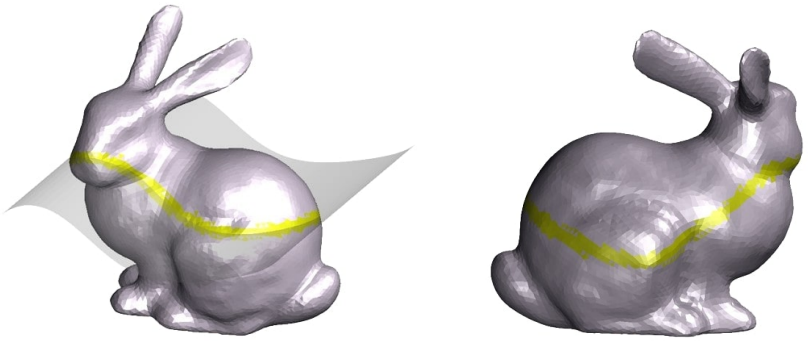


Figure 4. Generating a ruled surface.



(**a**) (**b**)

Figure 5. Detection of candidate faces (highlighted in yellow): (**a**) front view of Bunny model showing candidate faces and (**b**) back view of Bunny model showing candidate faces.

Let $T = \{\Delta_1, \Delta_2, \cdots, \Delta_n\}$ represent a set of candidate triangles. For each triangle Δ_i, we need to compute the exact intersection segment. We denote $l_j(t)$, where $j = 1, 2, 3$, as the parametric equations for the edges of triangle Δ_i. We then calculate the intersection points of $l_j(t)$ with $R(u, v)$. This task is commonly referred to as a Line–Surface Intersection (LSI) problem, for which various methods [20,21,23] have been devised to address it efficiently. In this paper, we focus on the LSI problem of a ruled surface with a line, as depicted in Figure 6a. Our LSI problem is decomposed into two simpler subproblems: (1) Plane–Curve Intersection (PCI) and (2) Line–Line Intersection (LLI), as follows:

1. PCI problem (Figure 6b)
   - Construct a plane π that contains $l(t)$ and the generator vector $\vec{r}$.
   - Determine the intersection point $C(u^*)$ between $C(u)$ and π.
2. LLI problem (Figure 6c)
   - Generate the ruling line $m(v)$ originating from $C(u^*)$ in the direction of $\vec{r}$.
   - Calculate the intersection point $R(u^*, v^*)$ between ruling line $m(v)$ and line $l(t)$.

In Figure 6, let $l(t) = \mathbf{p} + t\vec{d}$ denote the line segment of a triangle edge. A plane π passing through $\mathbf{p}$ with normal vector $\vec{n} = \vec{d} \times \vec{r}$ is intersected with $C(u)$ as shown in Figure 6b. The corresponding intersection point $C(u^*)$ can be found by solving the following equation:

$$< C(u) - \mathbf{p}, \vec{n} >= 0, \tag{4}$$

where $< \cdot, \cdot >$ represents an inner product. Equation (4) identifies the point lying simultaneously on the curve $C(u)$ and the plane π. Because Equation (4) involves only one unknown u, the corresponding intersection point can be computed more efficiently than the general LSI problem in Equation (2). Starting from the intersection point $C(u^*)$, we create a line $m(v) = C(u^*) + v\vec{r}$. As $m(v)$ has the direction of $\vec{r}$, it also lies on the plane π; thus, the final intersection point $R(u^*, v^*)$ can be found by solving a simple LLI problem between $l(t)$ and $m(v)$, formulated as the following linear system:

$$\begin{bmatrix} \vec{r} & \vec{d} \end{bmatrix} \begin{bmatrix} v \\ t \end{bmatrix} = \mathbf{p} - C(u^*). \tag{5}$$

Consequently, our method decomposes Equation (2) with two unknowns u and v into two simpler ones, that is, Equations (4) and (5), based on geometric interpretation. These simpler equations can be easily solved sequentially, enabling efficient and robust computation of intersection points. Algorithm 1 shows how to compute the intersection lines between a triangle mesh and a ruled surface using the proposed method.

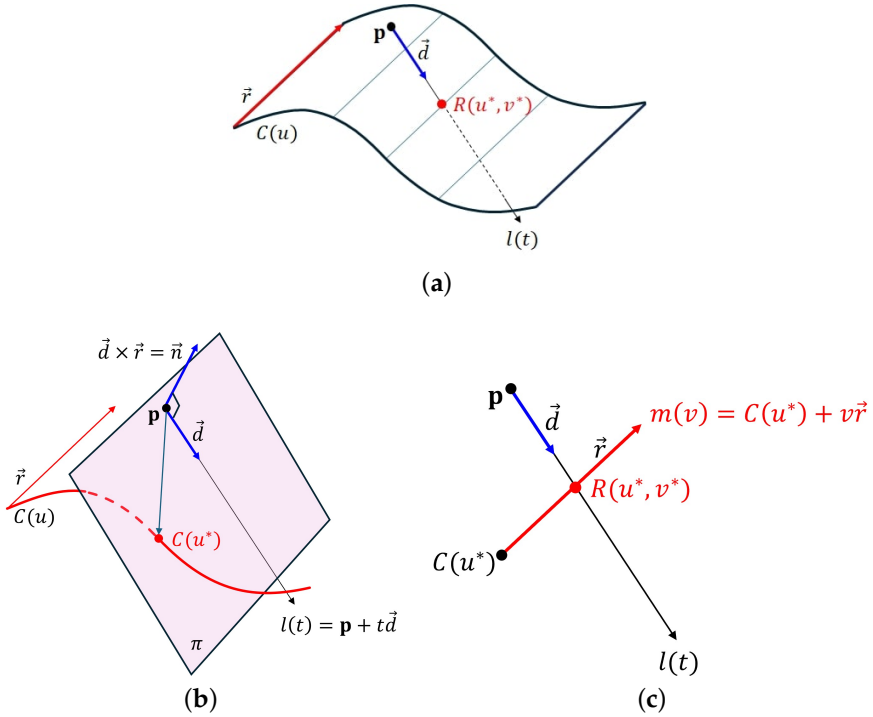


Figure 6. The LSI problem presented in (**a**) is broken down into two subproblems: (**b**) Plane–Curve Intersection (PCI) and (**c**) Line–Line Intersection (LLI).

Algorithm 1 Intersection of candidate triangles with a ruled surface

Require: $T = \{\Delta_i\}$: a set of candidate triangles, $R(u, v) = C(u) + v\vec{r}$: a ruled surface

for Δ_i **in** T **do**
  for e_j **in** Δ_i **do**
    $l_j(t) \leftarrow$ parametric line of e_j
    Create plane $\pi \leftarrow$ containing $l_j(t)$ and generator vector $\vec{r}$
    Compute intersection point $C(u^*)$ between π and $C(u)$
    $m(v) \leftarrow$ parametric line starting from $C(u^*)$
    Compute intersection point $R(u^*, v^*)$ between $l_j(t)$ and $m(v)$
  end for
end for

The next step is to create polylines to subdivide the mesh into several submeshes by connecting the previously computed intersection points. To achieve this, we categorize the potential intersections of a triangle with a ruled surface into two types: (1) single intersections and (2) double intersections, as depicted in Figure 7. It is necessary to prune the single intersections (Figure 7a) to remove the degenerate cut segments. When the degenerate segments have been removed, the remaining cut segments are connected to form a polyline. This polyline consists of the points on the edges, as the cut segments are produced by the intersection points with the ruled surface on the triangle's edges. Every triangle that includes a cut segment is refined and split into two or three smaller ones, creating submeshes (see Figure 8). As shown in Figure 8b, these newly created submeshes have open boundaries (holes). To make sure the submeshes are solid, cross-sections need to be generated to fill these holes, which will be explained in the next subsection.

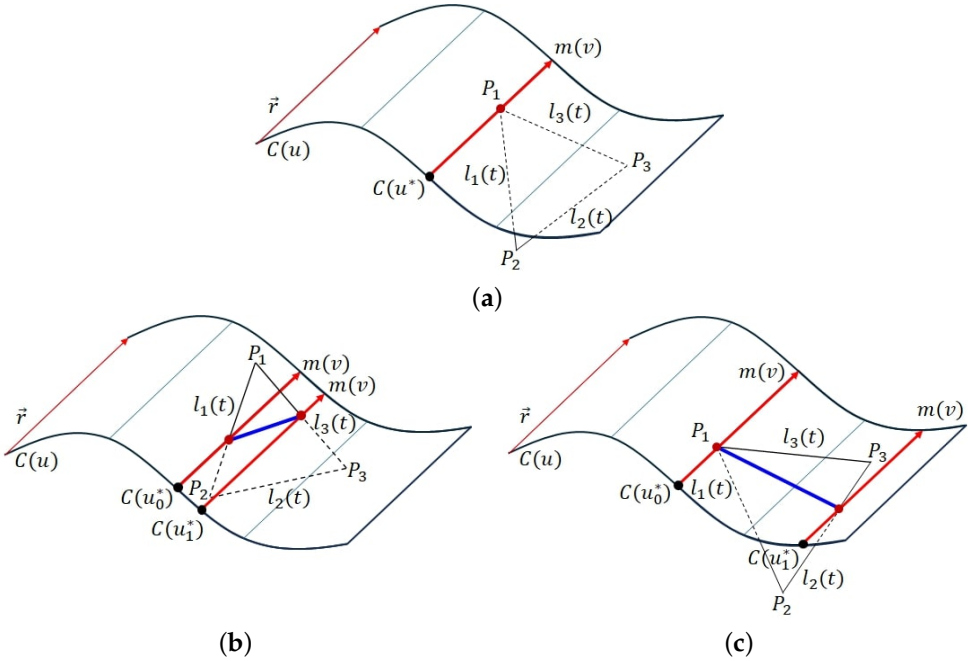


Figure 7. Triangle–Surface Intersections: (**a**) single intersection on a vertex, (**b**) two intersections on edges, and (**c**) two intersections on a vertex and an edge.

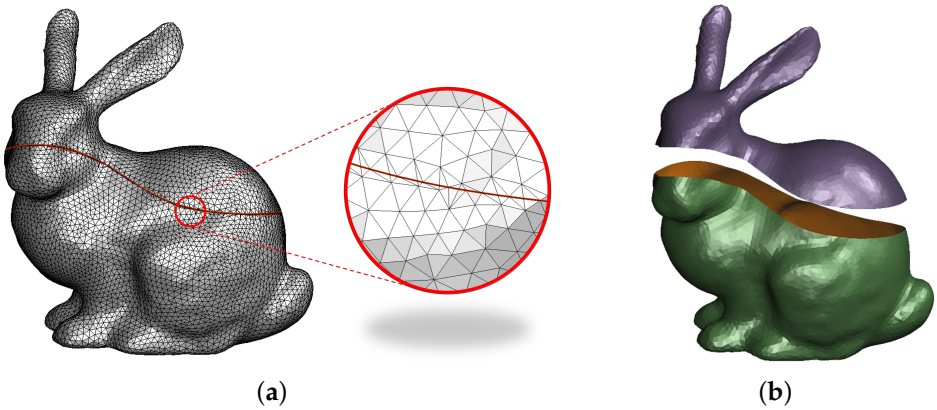


Figure 8. Submesh creation: (**a**) triangles are refined by intersection lines; (**b**) two submeshes with open boundaries.

3.3. Cross-Section Creation and Merging

The ruled slicing method described above can result in multiple submeshes with open boundaries (holes). While conventional hole-filling techniques [24–27] can be utilized to fill these holes, the resulting cross-sections may not align perfectly with those of the ruled surface. To address this issue and ensure that the cross-sections precisely match the ruled surface, it is imperative to generate the cross-section patch directly from the ruled surface. In our method, all intersection points have corresponding parameters (u^*, v^*) in the parametric domain $\Omega = [0,1] \times [0,1]$ of a ruled surface. By connecting these points we can easily create a polyline in Ω, as illustrated in Figure 9b. We uniformly sample points in Ω and create a subpatch $\phi(u,v)$ bounded by the polyline using the constrained Delaunay triangulation [28]. Figure 10a–d shows examples of the generated subpatch $\phi(u,v)$ with varying numbers of sampling points. The corresponding subpatch $X(u,v)$ of $R(u,v)$ can be defined as $X(u,v) = R \cdot \phi(u,v)$ and is utilized as the cross-section to fill the open boundary of a submesh, as shown in Figure 11.

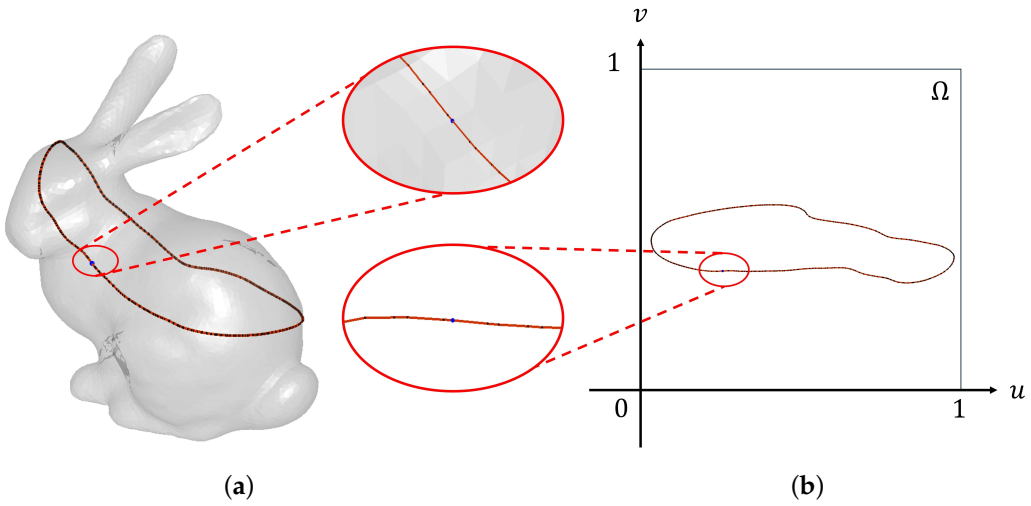


Figure 9. Cutting polyline in (**a**) 3D space and (**b**) the parametric domain Ω.

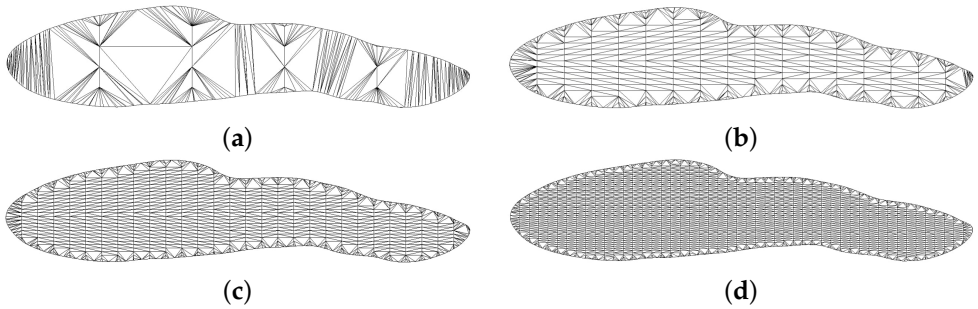


Figure 10. Subpatches $\phi(u,v)$ with different numbers of sampling points in u and v directions: (**a**) 6×6, (**b**) 18×18, (**c**) 30×30, (**d**) 50×50.

After a cross-section is created for each polyline in Ω, a merging step is necessary to ensure that the submeshes are solid. Merging the cross-section and submesh directly may lead to inconsistencies in orientation. To address this, we compare the boundary direction of the cross-section with that of the submesh. If both boundaries have the same direction, this indicates that the orientation of the cross-section is inconsistent with the submesh and must be flipped. If not, we simply merge them to make the submesh solid. Algorithm 2 illustrates the overall process of the proposed technique.



Figure 11. The subpatch of a ruled surface fills the cutting boundary of the Bunny model.

Algorithm 2 Mesh slicing algorithm

Require: Target mesh M, freeform curve $C(u)$, and generator vector $\vec{r}$
$R(u,v) \leftarrow$ ruled surface generated by $C(u)$ and $\vec{r}$
$T = \{\Delta_i\} \leftarrow$ set of candidate triangles of M intersected with BVH of $R(u,v)$
for Δ_i **in** T **do**
    Compute intersection segment s_i of Δ_i with $R(u,v)$ (Algorithm 1)
end for
$L = \{l_i\} \leftarrow$ set of polylines on M generated by connecting intersection segments
$\hat{L} = \{\hat{l}_i\} \leftarrow$ set of polylines in the parametric domain of $R(u,v)$
Subdivide $M = M_1 \cup M_2 \cup \cdots \cup M_n$ using l_i
Create cross sections $X = \{X_i\}$ by cutting $R(u,v)$ with $\hat{l}_i$
for M_i **in** M **do**
    for B_j **in** boundaries of M_i **do**
        Find the cross-section X_k that matches with B_j
        if normal of X_k is inconsistent with B_j **then**
            Flip normal of X_k
        end if
        Fill B_j by merging X_k with B_j
    end for
end for

4. Experimental Results

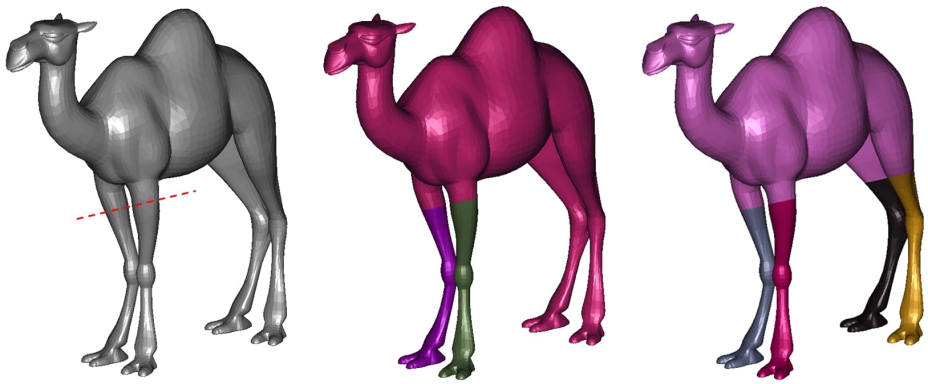
The proposed technique was implemented using the C++ language on a PC equipped with an Intel i7-9700F CPU running at 3.00 GHz, 40.0 GB of memory and an NVIDIA GeForce RTX 2060 graphics card. Mesh slicing experiments using a ruled surface were conducted with various 3D mesh models exhibiting different geometric characteristics. Table 1 lists the geometric details of the models used in the experiment.

Table 1. Statistics of the models.

Models	Box	Knot	Bunny	Camel	Armadillo	Pumpkin Creature
# of vertices	1538	9450	8327	9770	43,243	58,657
# of faces	3027	18,900	16,650	19,536	86,462	117,310
Feature	Sharp edges	Genus > 1	Complex Smooth surface	Complex Smooth surface	Complex High noise	Complex High noise

4.1. Mesh Slicing Results

Our method offers users two types of strokes: a straight line and a freeform curve. For a straight line, the user defines the start and end points of the cutting line by mouse clicking, dragging, and releasing. Subsequently, a ruled surface is generated from the B-spline curve that passes through two end points. It is important to note that although the generated ruled surface is essentially a plane, it is confined by the two end points; therefore, our method enables precise slicing of a desired region as shown in Figure 12. Various results of mesh slicing using straight lines are illustrated in Figure 13, where the blue dashed lines denote the user's strokes and the red solid lines correspond to the cutting polylines on the meshes. Note that the blue line may intersect with the mesh multiple times, resulting in multiple submeshes. Figure 14 shows the results of mesh slicing using freeform curves, where the curves intersect with the target meshes once, resulting in two submeshes, respectively. Our method allows a freeform curve to intersect with the target mesh multiple times, generating multiple submeshes, as shown in Figure 15 (refer to the attached video for a real-time demonstration (https://youtu.be/UqQaKZTj_Zo (accessed on 23 April 2024) or see the Video S1).



(**a**) (**b**) (**c**)

Figure 12. Comparison of slicing results between (**b**) our method and (**c**) conventional plane for the same user's stroke in (**a**).

The user's stroke is approximated by a freeform curve, then the target mesh is sliced by the ruled surface generated from the freeform curve. However, certain users may prefer more sophisticated control over the slicing result by editing the generated freeform curve. To accommodate this, our method provides the flexibility for users to modify the slicing curve. Figure 16 illustrates an example of manual editing of the generated freeform curve. As the user draws the slicing trajectory on the screen, the initial approximation curve is generated. By adjusting the control points (the green circle), the curve is modified as the user intends.

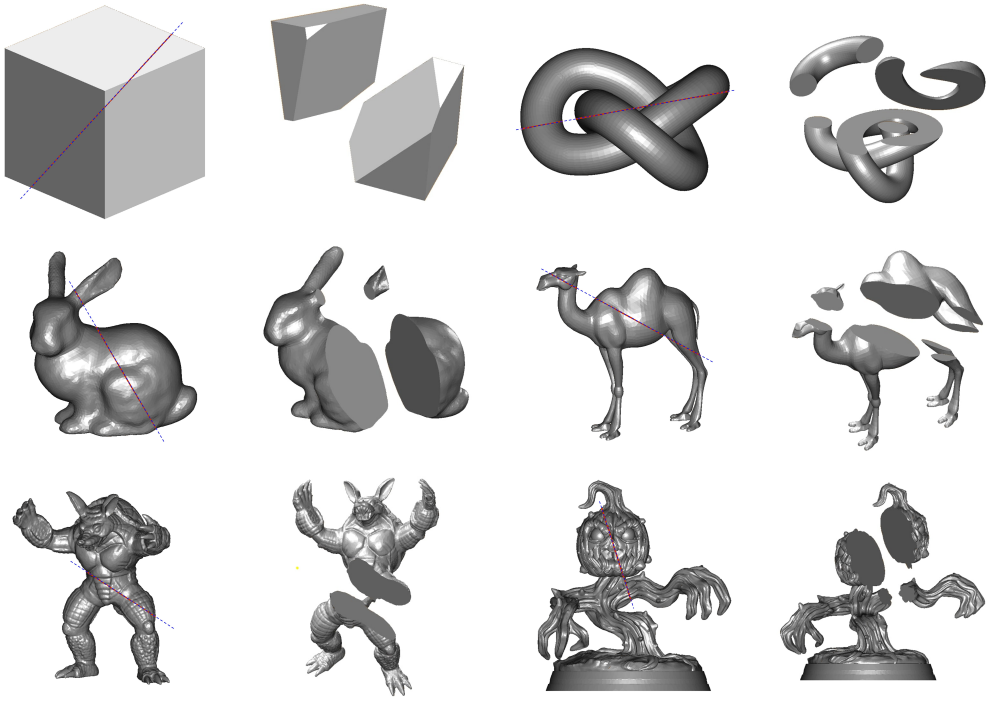


Figure 13. Various mesh slicing results using straight line strokes.

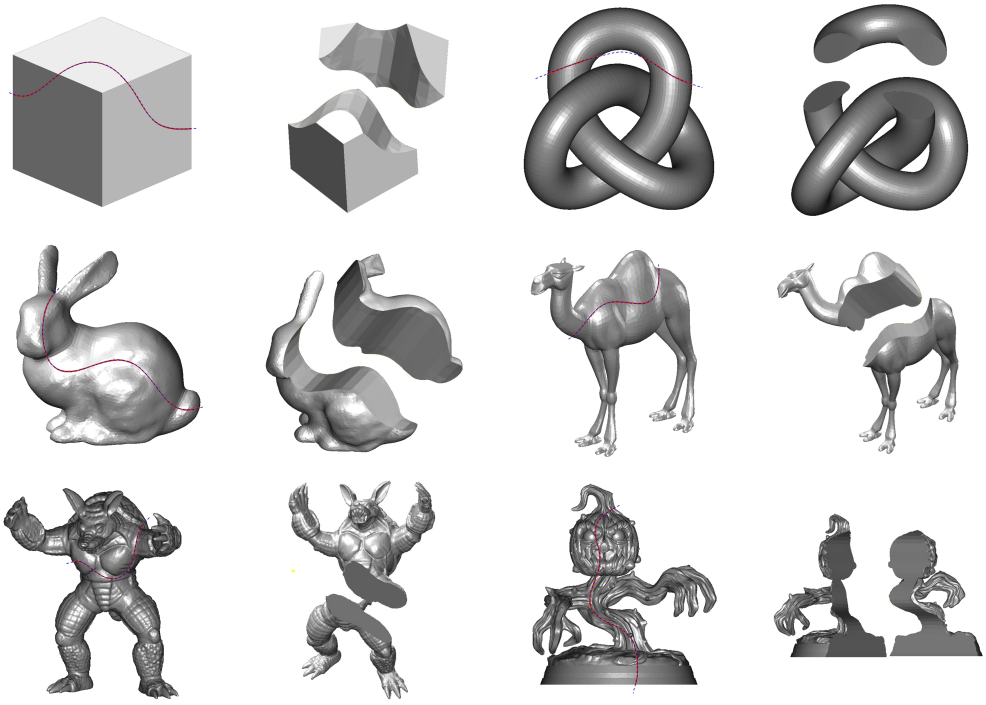


Figure 14. Various results of slicing meshes into two submeshes using freeform curves.

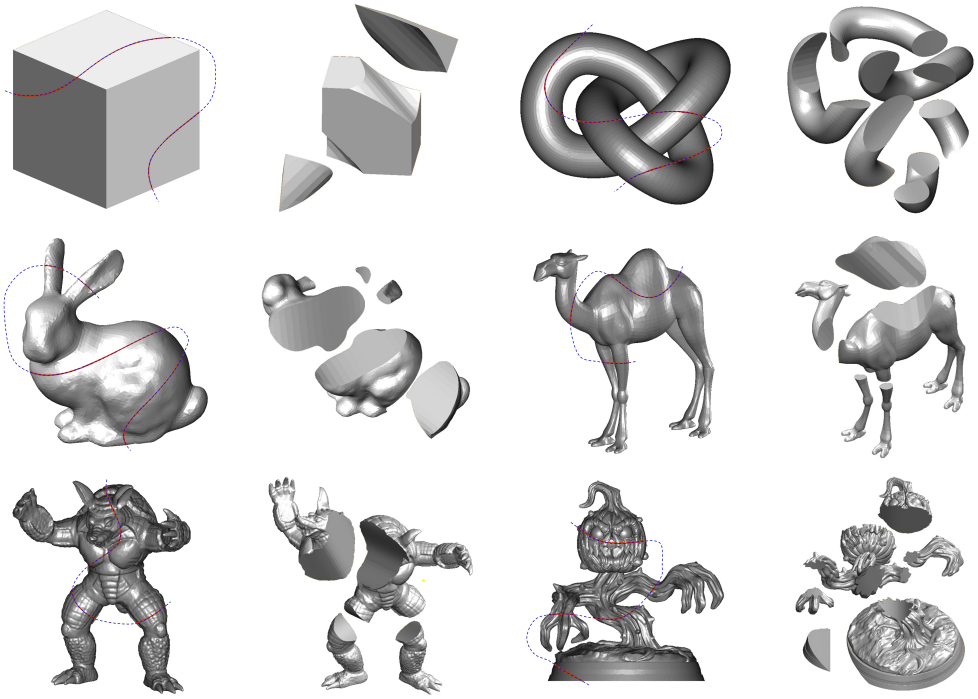


Figure 15. Various results of slicing meshes into multiple submeshes using freeform curves.

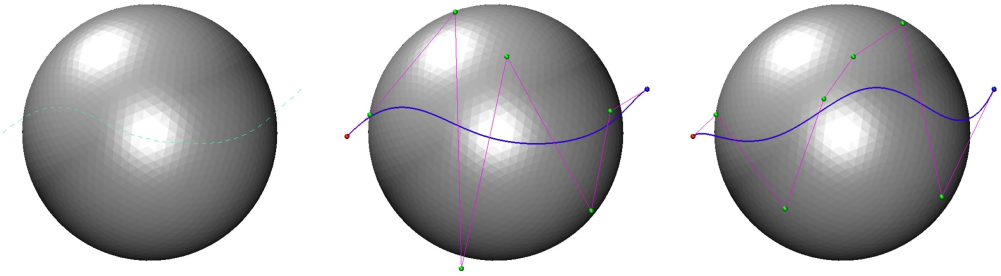


Figure 16. Editing slicing curve: (**a**) user's free stroke, (**b**) initial approximation B-spline curve, and (**c**) editing control points of a slicing curve.

4.2. Performance Analysis

Table 2 lists the elapsed times for slicing various meshes into submeshes, corresponding to each step of the slicing process depicted in Figures 13–15. The elapsed time for each step is then summed to determine the total elapsed time. Regardless of the diverse features among the models, the majority of the elapsed time is measured on the intersection computation, submesh creation, and merging steps. Even in the case of models such as the Armadillo and Pumpkin Creature, which have significantly larger numbers of vertices and faces, most experiments yielded satisfactory results within approximately 1 to 2 s. This observation suggests that the elapsed time is primarily influenced by the number of candidate faces listed in the third row, rather than the complexity of the model.

To demonstrate its computational efficiency, we compared the elapsed times of our method (decomposition into PCI and LLI) with a general LSI computation in Equation (2). We performed mesh slicing 100 times on various models and computed the average elapsed time for each method. Figure 17 shows the comparative results of the two methods. The blue line corresponds to our method and the orange line to general LSI computation. The elapsed time of our method is significantly less than that of the general LSI method. Moreover, the difference between the two methods becomes larger as the number of candidate faces

increases. This confirms that our method is much faster and more efficient than the general LSI method.

Table 2. Elapsed time (in ms) of mesh slicing algorithm.

Models	Box	Knot	Bunny	Camel	Armadillo	Pumpkin Creature
Ruled surface creation	0	0	0	0	0	0
# of Candidate Faces	387	609	457	578	1170	3050
Intersection computation	73	80	88	159	230	499
Submesh creation	21	84	32	133	530	1183
Cross-section creation	2	2	2	4	4	12
Normal correction	1	3	1	3	7	14
Merging cross-section	13	41	19	87	298	506
Total	110	210	142	386	1069	2214

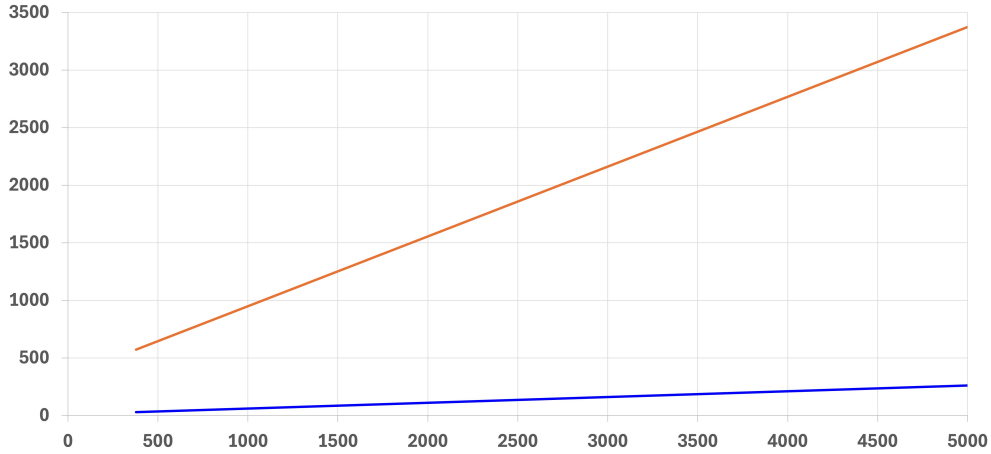


Figure 17. Efficiency comparison between the LSI and our methods. The horizontal axis represents the number of candidate faces and the vertical axis represents the elapsed time (ms).

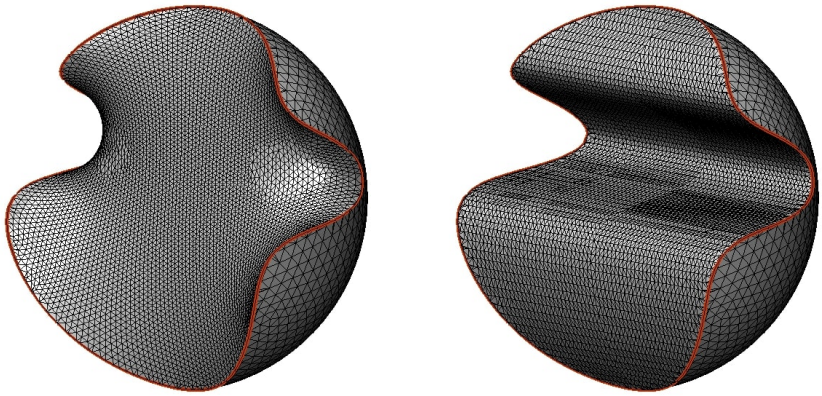
4.3. Comparison to Existing Methods

To demonstrate the effectiveness of our method for generating cross-sections, we compared our method with the state-of-the-art surface fairing technique from [24], which fills holes with a minimal surface while keeping its boundary fixed. For the user's identical strokes, Figure 18 shows the cross-sections generated by the minimal surface and our method, respectively. As shown in Figure 18a, the hole-filling method based on the surface fairing is not suitable for the desired cross-section, whereas our method produces the cross-section by directly cutting the ruled surface. Therefore, as shown in Figure 18b, it exactly matches the slicing cross-section. This effectiveness highlights the advantage of our method, which allows for the generation of accurate cross-sections with arbitrary boundary configurations.

We additionally compared the mesh slicing results by discretizing the freeform curve into a set of line segments and directly computing the intersection points between the curve and the triangular mesh. Figure 19 demonstrates the effectiveness of our method. Figure 19b,c shows excessive complexity and unnecessary refined triangles due to the refinement of the triangles at the endpoints of the line segments. On the other hand, our method refines the triangles only at the intersection points while avoiding unnecessary refinements, and produces concise and optimal triangles, as shown in Figure 19a.

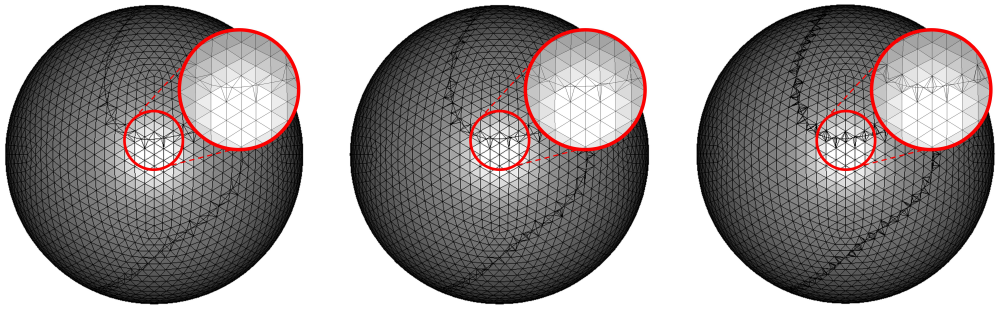
To quantitatively compare the quality of the slicing results generated using our method and the discretization method, we sliced each model in Table 1 using an identical freeform curve. Table 3 lists the number of intersection points (vertices) and degenerate triangles at the boundaries of the cross-sections generated by our method and the discretization method with different numbers of sample points. The discretization method generates more intersection points and degenerate triangles as the number of sample points increases,

resulting in higher geometric complexity of the cross-sections, as shown in Figure 19b,c. Conversely, our method produces fewer intersection points and degenerate triangles, resulting in more effective and robust slicing results.



(**a**) (**b**)

Figure 18. Comparison of cross-section generation results using (**a**) surface fairing method and (**b**) our method.



(**a**) (**b**) (**c**)

Figure 19. Mesh slicing results: (**a**) computing intersection points, (**b**) using a set of 30 line segments, and (**c**) using a set of 100 line segments.

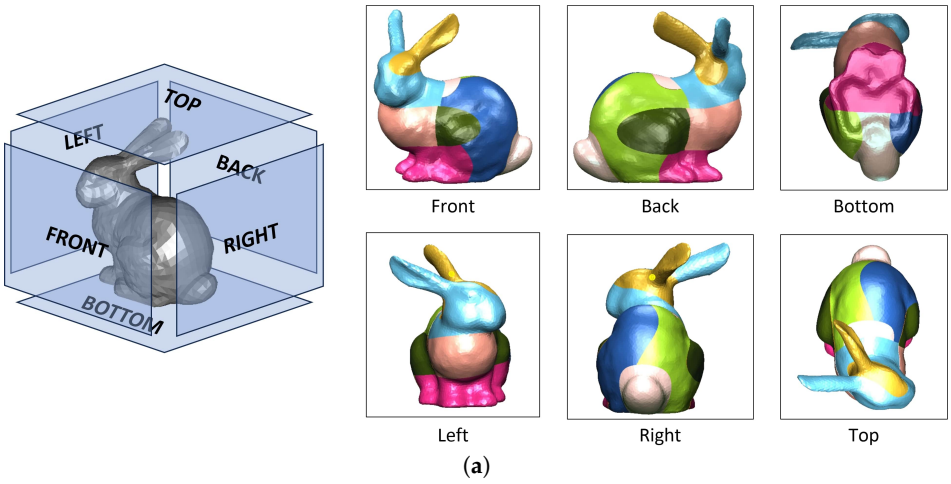
Table 3. Comparison of the number of intersection points and degenerate triangles between our method and the discretization method (triangles with an area less than $\epsilon = (10^{-3})$ are classified as degenerate triangles).

Models	# of Intersection Points			# of Degenerate Faces		
	Our	Discretization (30 Samples)	Discretization (100 Samples)	Our	Discretization (30 Samples)	Discretization (100 Samples)
Box	207	343	661	0	8	12
Knot	447	583	911	0	12	18
Bunny	365	470	715	4	21	38
Camel	373	494	795	0	2	16
Armadillo	859	973	1238	5	13	10
Pumpkin Creature	1182	1320	1683	33	54	60

4.4. Application

Our method can be easily applied in various applications, such as custom 3D puzzle generation. As an example, we sliced the Bunny model into eight pieces, as shown in Figure 20. Figure 20a shows the results of slicing the Bunny model, while Figure 20b shows

the 3D printed puzzle pieces using a fused deposition modeling (FDM) method, where the melted filament is stacked layer-by-layer to form the 3D model. While this process can introduce slight errors, the resulting pieces still provide satisfying results. When assembled (Figure 20c), these pieces perfectly reconstruct the original Bunny model. Expanding this application to the manufacturing industry could facilitate the creation of even more intricate and highly sophisticated 3D puzzles.



(**a**)



(**b**) (**c**)

Figure 20. Application to 3D puzzle scenario: (**a**) mesh slicing results using our method, (**b**) 3D printed puzzle pieces, and (**c**) reconstructed 3D model.

5. Conclusions and Discussion

This paper introduces an efficient method for slicing a triangle mesh using a freeform curve. The method utilizes a ruled surface generated based on the user's stroke, and has been successfully tested on various triangle meshes with different characteristics. The user interaction involves drawing a stroke on the screen, which is then approximated by a B-spline curve. Subsequently, the curve is extruded in the viewing direction, creating a ruled surface. To achieve efficient slicing, we leverage the BVH of the ruled surface to identify candidate faces for potential intersections. When the candidate faces have been identified, we compute the exact intersection points between the ruled surface and these faces. This process is efficiently performed by decomposing the LSI problem into PCI and LLI problems. This decomposition allows for efficient computation of the intersection parameters u and v, improving both accuracy and robustness. Intersection points are then connected to form polylines, effectively cutting the mesh into multiple submeshes. To ensure the solidity of the submeshes, cross-sections are generated by trimming the ruled surface along the polylines, then merged with the corresponding submeshes. We have demonstrated the effectiveness of the proposed method by comparing it with existing methods and showing various examples of mesh slicing.

Our approach offers several advantages. First, our method enables users to slice triangle meshes along arbitrary trajectories, encompassing both straight and freely curved paths by editing the freeform curves. Second, unlike existing methods that limit mesh slicing to two pieces per stroke, our approach allows for slicing the mesh into multiple submeshes regardless of the mesh topology. The decomposition of LSI into PCI and LLI further improves the efficiency and robustness of the method by enabling separate computations of intersection parameters. Finally, our cross-sections can be easily generated from the ruled surface, while existing methods rely on a hole-filling method.

Despite the numerous advantages mentioned above, the current implementation is not able to handle volumetric meshes. This poses a significant challenge, as volume elements such as tetrahedra involve a considerably greater number of intersection points compared to triangles, leading to an exponential increase in computational cost. In future work, we aim to enhance the efficiency of our technique by incorporating GPU-based parallel programming, enabling real-time slicing of volumetric meshes.

Supplementary Materials: The following supporting information can be downloaded at: https://www.mdpi.com/article/10.3390/math12101432/s1, Video S1: Experimental_Results.

Author Contributions: S.-Y.L. and S.-H.K. conceived and designed the experiments; S.-Y.L. implemented the proposed technique and performed the experiments; S.-H.Y. wrote the paper. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) under the Artificial Intelligence Convergence Innovation Human Resources Development (IITP-2024-RS-2023-00254592) grant funded by the Korea government (MSIT).

Data Availability Statement: Data are contained within the article.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Khan, D.; Plopski, A.; Fujimoto, Y.; Kanbara, M.; Jabeen, G.; Zhang, Y.J.; Zhang, X.; Kato, H. Surface remeshing: A systematic literature review of methods and research directions. *IEEE Trans. Vis. Comput. Graph.* **2020**, *28*, 1680–1713. [CrossRef]
2. Durand, R.; Pantoja-Rosero, B.; Oliveira, V. A general mesh smoothing method for finite elements. *Finite Elem. Anal. Des.* **2019**, *158*, 17–30. [CrossRef]
3. Salinas, D.; Lafarge, F.; Alliez, P. Structure-aware mesh decimation. In *Computer Graphics Forum*; Wiley Online Library: Hoboken, NJ, USA, 2015; Volume 34, pp. 211–227.
4. Selim, M.; Koomullil, R. Mesh deformation approaches—A survey. *J. Phys. Math.* **2016**, *7*, 1–9.
5. Lee, W.B.; Hao, W.Y.; Kyung, B.P.; Ryu, S.H. Dismantling Simulation of Nuclear Reactor Using Partial Mesh Cutting Method for 3D Model. *J. Digit. Converg.* **2015**, *13*, 303–310.
6. Mitani, J. A simple-to-implementation method for cutting a mesh model by a hand-drawn stroke. In Proceedings of the EUROGRAPHICS Workshop on Sketch-Based Interfaces and Modeling, Trinity College Dublin, Dublin, Ireland, 28–29 August 2005.
7. Igarashi, T.; Matsuoka, S.; Tanaka, H. Teddy: A sketching interface for 3D freeform design. In *ACM SIGGRAPH 2006 Courses*; Association for Computing Machinery: New York, NY, USA, 2006; p. 11-es.
8. Owada, S.; Nielsen, F.; Okabe, M.; Igarashi, T. Volumetric illustration: Designing 3D models with internal textures. In *ACM SIGGRAPH 2004 Papers*; Association for Computing Machinery: New York, NY, USA, 2004; pp. 322–328.
9. Elber, G.; Kim, M.S. Geometric constraint solver using multivariate rational spline functions. In Proceedings of the Sixth ACM Symposium on Solid Modeling and Applications, New York, NY, USA, 4–8 June 2001; pp. 1–10. [CrossRef]
10. Chu, C.W.; Kim, K.K.; Park, C.J.; Choi, J.S. An Interactive 3D Mesh Editing System for Printing Object Larger Than the Printing Volume of 3D Printer. *J. Korea Multimed. Soc.* **2016**, *19*, 1619–1625. [CrossRef]
11. Kang, Y.; Kim, S. Three-dimensional garment pattern design using progressive mesh cutting algorithm. *Int. J. Cloth. Sci. Technol.* **2019**, *31*, 339–349. [CrossRef]
12. Owada, S.; Nielsen, F.; Nakazawa, K.; Igarashi, T. A sketching interface for modeling the internal structures of 3d shapes. In *ACM SIGGRAPH 2006 Courses*; Association for Computing Machinery: New York, NY, USA, 2006; p. 12-es.
13. Khan, L.; Choi, Y.J.; Hong, M. Cutting simulation in unity 3D using position based dynamics with various refinement levels. *Electronics* **2022**, *11*, 2139. [CrossRef]
14. Kamarianakis, M.; Protopsaltis, A.; Angelis, D.; Tamiolakis, M.; Papagiannakis, G. Progressive tearing and cutting of soft-bodies in high-performance virtual reality. *arXiv* **2022**, arXiv:2209.08531.
15. Caligiana, P.; Liverani, A.; Ceruti, A.; Santi, G.M.; Donnici, G.; Osti, F. An interactive real-time cutting technique for 3D models in mixed reality. *Technologies* **2020**, *8*, 23. [CrossRef]

16. Lu, C.; Lin, Y.; Ji, Z.; Dalian Univ. of Technology. An algorithm for plane-surface intersection and its application to shipbuilding. *Ship Technol. Res.* **2005**, *52*, 103–106. [CrossRef]
17. Sharma, R.; Sha, O. A tracing method for parametric Bezier triangular surface/plane intersection. *Int. J. Comput. Appl. Technol.* **2007**, *28*, 240–253. [CrossRef]
18. Li, X.; Jiang, S.; Huang, B.; Zeng, Q. An Efficient Method for Calculating the Intersection Curve between a Ruled Surface and a Plane. *Int. J. Signal Process. Image Process. Pattern Recognit.* **2013**, *6*, 331–340. [CrossRef]
19. Beer, G. Algorithms for geometrical operations with NURBS surfaces. *arXiv* **2022**, arXiv:2210.13160.
20. William, M.; Elaine Cohen, R.F.; Shirley, P. Practical Ray Tracing of Trimmed NURBS Surfaces. *J. Graph. Tools* **2000**, *5*, 27–52. [CrossRef]
21. Laurent, B. Implicit matrix representations of rational Bézier curves and surfaces. *Comput.-Aided Des.* **2014**, *46*, 14–24. [CrossRef]
22. Shen, J.; Busé, L.; Alliez, P.; Dodgson, N. A line/trimmed NURBS surface intersection algorithm using matrix representations. *Comput. Aided Geom. Des.* **2016**, *48*, 1–16. [CrossRef]
23. Wang, X.; Zhang, W.; Huang, X. Computation of point inversion and ray-surface intersection through tracing along the base surface. *Vis. Comput.* **2015**, *31*, 1487–1500. [CrossRef]
24. Feng, C.; Liang, J.; Ren, M.; Qiao, G.; Lu, W.; Liu, S. A fast hole-filling method for triangular mesh in additive repair. *Appl. Sci.* **2020**, *10*, 969. [CrossRef]
25. Park, J.H.; Park, S.; Yoon, S.H. Parametric blending of hole patches based on shape difference. *Symmetry* **2020**, *12*, 1759. [CrossRef]
26. Jun, Y. A piecewise hole filling algorithm in reverse engineering. *Comput.-Aided Des.* **2005**, *37*, 263–270. [CrossRef]
27. Zhao, W.; Gao, S.; Lin, H. A robust hole-filling algorithm for triangular mesh. *Vis. Comput.* **2007**, *23*, 987–997. [CrossRef]
28. Gaur, P.K.; Bose, S.K. On recent advances in 2D constrained Delaunay triangulation algorithms. *arXiv* **2017**, arXiv:1707.05949.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.